

Week 1 Internship Report

CityScape – Mini City Builder

Intern Name: Nathaniel Sandipurti

Project Title: CityScape – Mini City Builder

Week: 1

Duration: Day 1 – Day 6

Github Folder: <https://github.com/Nathaniel06/CityScape>

Objective

This week was about turning a rough idea into something I could actually build on. My goal was to lock down the concept, get Unity set up properly, gather and organize the assets I'd need, and get the core foundation of a city-building game in place — terrain, roads, grid-based building placement, and a basic UI.

Daily Work Log

Day 1 – Project Planning and Research

- Spent time exploring different game ideas that would work well for the internship timeline.
- Settled on CityScape – Mini City Builder as the project.
- Mapped out the scope and thought through the core gameplay mechanics I wanted to include.
- Looked at a few existing city-building games for inspiration and noted down the systems that seemed essential.

Outcome: Had a finalized concept and a rough development roadmap to work from.

Day 2 – Asset Collection

- Sourced a set of low-poly environment assets.
- Imported residential, industrial, commercial, utility, decoration, road, and terrain assets into Unity.
- Sorted everything into clearly named folders so the project wouldn't turn into a mess later.

Outcome: A usable asset library and a project structure that made sense.

Day 3 – Project Structure and Environment Setup

- Built the main gameplay scene.
- Blocked out the map using terrain, mountains, trees, and other environment pieces.
- Sketched out the overall city layout.
- Planned the architecture for the systems I'd need going forward: grid system, road placement, building placement, camera controller, UI structure, economy manager, and save/load system.

Outcome: The project structure was in place and ready for actual gameplay code.

Day 4 – Core Gameplay Implementation

- Got the grid-based placement system working.
- Built out road placement functionality.

- Added building placement with grid snapping.
- Implemented object rotation before placing objects down.

Outcome: Basic city construction was functional for the first time.

Day 5 – UI and Gameplay Features

- Built the building category panel.
- Added Residential, Industrial, Utility, Services, Decoration, and Road menus.
- Added a building information panel.
- Implemented camera controls.
- Added Build and Explore modes.

Outcome: A working gameplay interface came together.

Day 6 – Game Systems Integration and Testing

- Implemented save and load functionality.
- Added a pause menu.
- Put together a basic economy system.
- Added a minimap.
- Tested everything and fixed a handful of placement bugs.
- Cleaned up the scene layout and UI.

Outcome: The first playable prototype was complete.

Technologies Used

- Unity 6
- C#
- Unity UI System
- Git & GitHub
- Git LFS
- Low-Poly Asset Packs

Features Completed

- Low-poly environment setup
- Terrain and environment design
- Road construction system
- Grid-based building placement
- Building rotation
- Residential buildings
- Industrial buildings
- Utility buildings
- Service buildings
- Decoration system
- Build and Explore modes
- Pause menu
- Save/Load system

- Minimap
- Building information panel
- Basic economy UI

Challenges Faced

- Figuring out a project scope that was realistic for the internship's timeframe.
- Keeping a large number of imported assets organized.
- Designing a grid placement system that could actually scale.
- Wiring up multiple gameplay systems together without the project turning messy.

Solutions Implemented

- Trimmed the scope down to focus on the core city-building mechanics.
- Organized assets into categorized folders from the start.
- Used a modular architecture so systems could be built and tested independently.
- Built and tested each feature on its own before integrating it with the rest.

Conclusion

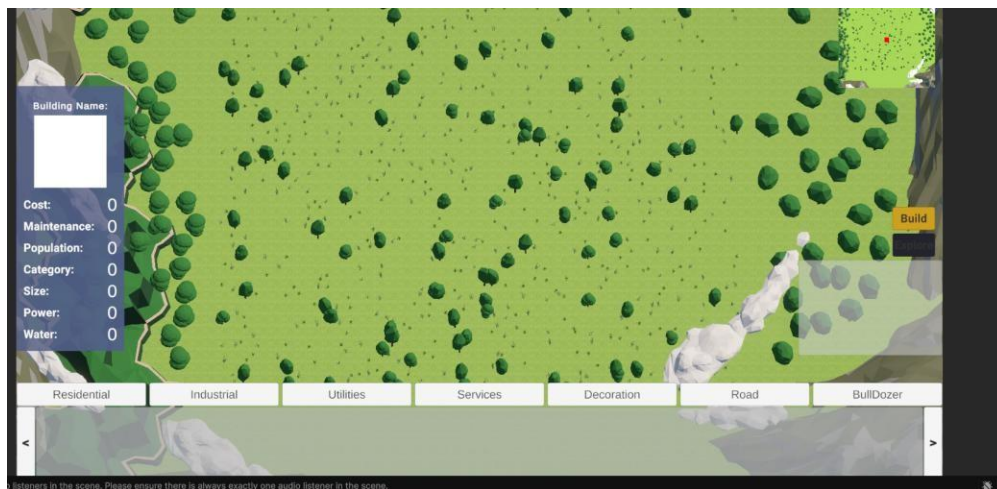
By the end of the first week, CityScape had gone from a bare idea to a working prototype. There's now a structured environment, grid-based building placement, road construction, categorized building menus, save/load functionality, and a basic UI in place. It's a solid base to build on for city simulation mechanics, resource management, and more advanced gameplay features in the coming weeks.

Screenshot Reference

A few screenshots taken along the way, showing how the project moved from an early prototype to a more complete build.



Day 1 — early road-building prototype, with placeholder UI buttons for roads, factories, shops, and houses.



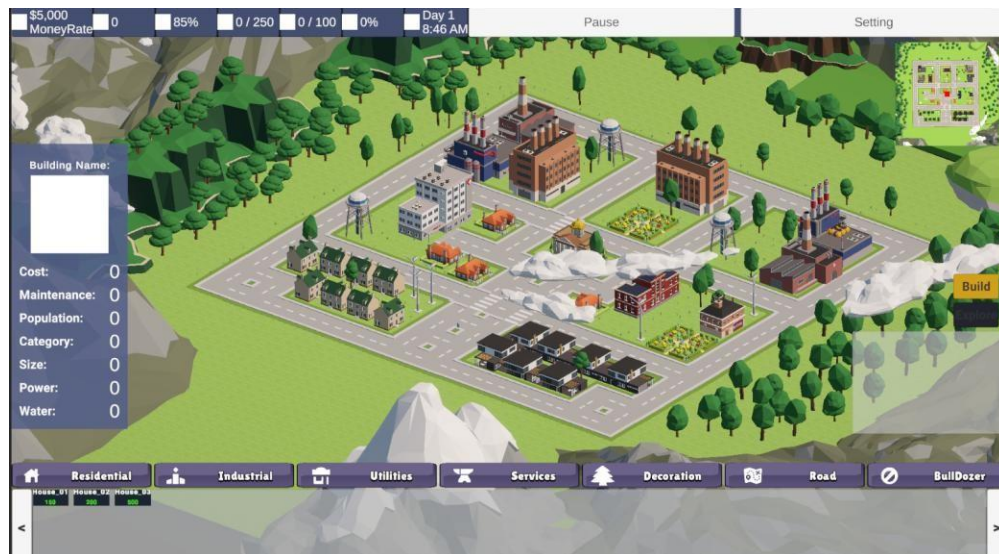
Day 2 — terrain and environment setup, with the building info panel and category tabs taking shape.



Day 3 — structured map with , money system, and category menu in place.



Day 4-5 — building placement in action, with rotation controls and per-category costs visible.



Day 6 — the integrated prototype: a fully laid-out city with roads, zoned buildings, and a detailed minimap.

Expected Deliverables

For this submission, the following needs to be included:

- A Unity project folder containing a scene with at least one terrain.
- A few imported 3D models (free assets are acceptable).
- Proper asset organization in folders.
- A README file explaining the project structure and the overall approach, submitted in PDF format.

Key Steps

- Open the Project
- Click on Assets\ Scene\ SampleScene
- After Open that Scene Click Game Window Play Button
- Click Pause UI Button
- Then Click Load
- Map will be load which I made during my 1 Week of Unity Game Development Trainee Internship

Week 2 Internship Report

CityScape – Mini City Builder

Intern Name: Nathaniel Sandipurti

Project Title: CityScape – Mini City Builder

Week: 2

Duration: Day 7 – Day 14

Github Folder: <https://github.com/Nathaniel06/CityScape>

Objective

The objective of Week 2 was to implement a functional player controller and basic physics interactions within the existing CityScape project.

During Week 1, the core city-building prototype was developed with terrain, roads, buildings, a grid-based placement system, and Build and Explore modes. The existing Week 1 prototype also included camera controls, a minimap, economy UI, and save/load functionality.

For Week 2, I focused on improving the Explore Mode by allowing the player to directly move around and explore the created city. A basic player movement system was implemented using keyboard controls along with a jumping mechanic.

Work Completed

During this week, I implemented a basic player controller for the Explore Mode of CityScape.

The player can move around the city using the W, A, S, and D keys. This allows the player to navigate through the environment and explore the city from a ground-level perspective.

A jumping mechanic was also implemented. The player can press the Spacebar to jump while exploring the city.

The movement system was integrated with the existing Explore Mode instead of creating a separate player controller demonstration scene. This helped maintain the overall structure and concept of the CityScape project.

The existing city environment, including roads, buildings, terrain, and other environment objects, was used as the testing area for player movement.

Player Controls

Input	Action
W	Move Forward
A	Move Left
S	Move Backward
D	Move Right
Spacebar	Jump

Explore Mode Implementation

The Explore Mode allows the player to move around the city after interacting with the city-building environment.

When Explore Mode is active, the player can control the character using keyboard input.

The player can walk through the city environment and inspect the roads, buildings, and overall city layout from a closer perspective.

This creates a connection between the city-building system and player exploration.

The Explore Mode adds an interactive gameplay element to CityScape instead of limiting the game to only a top-down city-building view.

Player Movement System

The player movement system was implemented using a custom C# script.

Keyboard input is used to detect movement directions.

The movement input from the W, A, S, and D keys is converted into a movement direction for the player.

The player character responds to the input and moves through the Unity scene.

The movement system was tested within the existing CityScape environment to ensure that the player could navigate around the city.

Jumping System

A basic jumping system was implemented as part of the player controller.

When the player presses the Spacebar, the player character performs a jump.

The jumping mechanic adds vertical movement to the player controller and allows basic interaction with the physics environment.

The jump system was tested on the terrain and around different areas of the city.

Basic Physics

Basic Unity physics were used to support the player movement and jumping mechanics.

The player character interacts with the environment while moving through the scene.

Gravity allows the player to return to the ground after jumping.

Collision components are used within the environment to define physical objects and surfaces.

The city terrain acts as the primary ground surface for the player.

The existing buildings and environment objects provide a practical environment for testing player movement and physics interactions.

Integration with CityScape

Instead of developing the player controller as an independent Unity scene, I integrated it directly into the existing CityScape project.

The Week 1 project already contained a complete prototype with roads, zoned buildings, and a detailed minimap, as shown in the integrated prototype screenshot on page 5 of the Week 1 report.

The player controller extends this prototype by making Explore Mode interactive.

The current gameplay structure is:

Build Mode → Build the City → Explore Mode → Walk Around the City

This integration allows the player to experience the city they created from a different perspective.

Technologies Used

- Unity 6
- C#
- Unity Physics System
- Player Movement Scripting
- Collision Components
- Unity Terrain
- Git and GitHub

Features Completed

- Explore Mode player movement
- WASD keyboard movement
- Forward movement
- Backward movement
- Left and right movement
- Player jumping
- Spacebar jump input
- Basic gravity interaction
- Player movement inside the CityScape environment
- Integration with the existing CityScape project
- Ground-level city exploration

Challenges Faced

One of the main challenges was integrating player movement into an existing city -building project.

The CityScape project was originally focused mainly on grid-based building placement and city construction. The Week 1 report describes the project foundation as a combination of road construction, grid-based placement, categorized building menus, save/load functionality, and UI systems.

Adding direct player movement required ensuring that the Explore Mode worked alongside the existing gameplay systems.

Another challenge was implementing jumping and movement in a way that felt responsive inside the existing terrain environment.

Solutions Implemented

The player controller was integrated specifically with Explore Mode.

The existing CityScape scene was used as the player movement and physics testing environment.

The movement controls were kept simple using WASD input so that the player could easily navigate the city.

A Spacebar-based jumping system was added to provide basic vertical movement.

The movement and jumping systems were tested directly inside the city environment instead of using a separate test project.

Testing

The player controller was tested inside the CityScape gameplay scene.

The following functionality was tested:

- WASD movement
- Forward and backward movement
- Left and right movement
- Jumping using the Spacebar
- Player movement on the terrain
- Player exploration around city roads
- Player movement near buildings
- Explore Mode gameplay

Testing helped identify movement and interaction issues while navigating the city environment.

Conclusion

During Week 2, I implemented a functional player controller and integrated it with the Explore Mode of CityScape.

The player can now use WASD controls to move around the city and press the Spacebar to jump.

This feature improves the gameplay experience by allowing the player to directly explore the city environment instead of only interacting with the game through the city-building interface.

The implementation also fulfills the core objective of the Week 2 internship task by demonstrating keyboard-based player movement, jumping, and basic physics interaction within a playable Unity scene.

The CityScape project now combines city building and direct city exploration, creating a stronger foundation for future gameplay systems.

Screenshot Reference

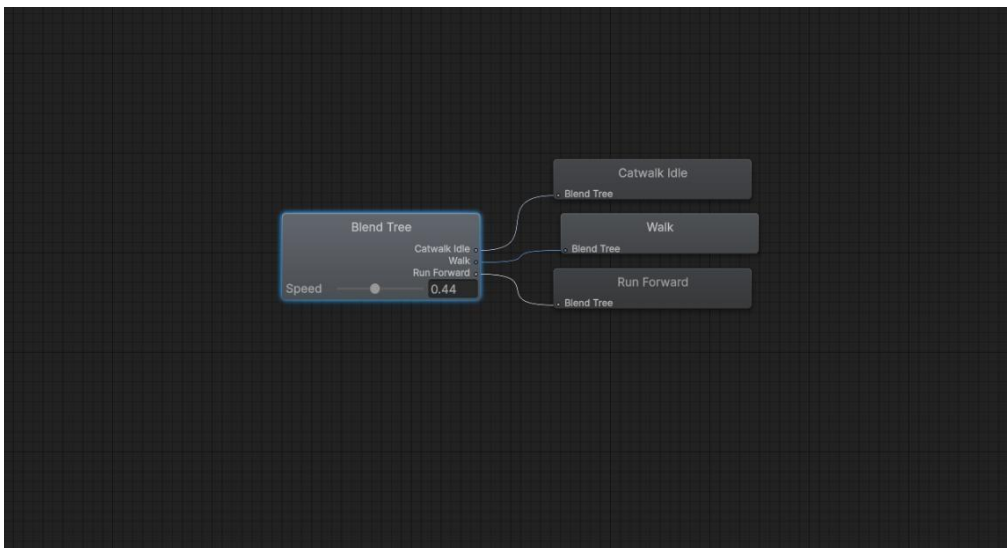
A few screenshots taken along the way, showing how the project moved from an early prototype to a more complete build.



Day 7 — Finalizing of Character which will be the Lady Mayor



Day 8 — Applying animations From Mixamo



Day 9-10 — Using blend Tree for integrating the animation in player



Day 11-12 — Implement the animation with movement and debug the errors



Day 13-14 — the integrated prototype: a fully laid-out city with roads, zoned buildings, and a detailed minimap where Player can roam.

Week 3 Internship Report

CityScape – Mini City Builder

Intern Name: Nathaniel Sandipurti

Project Title: CityScape – Mini City Builder

Week: 3

Duration: Day 15 – Day 21

GitHub Folder: <https://github.com/Nathaniel06/CityScape>

Objective

The objective of Week 3 was to make the city feel more alive by implementing a dynamic NPC (Non-Player Character) system within the existing CityScape project.

During Week 1, the core city-building mechanics were developed, including terrain generation, road placement, building placement, UI, and economy systems. Week 2 introduced an Explore Mode where the player could walk around the city using a third-person character.

For Week 3, I focused on adding life to the city by implementing NPC spawning, autonomous movement, and basic player interaction. NPCs now appear as the city develops, walk along the road network, and can be interacted with while exploring the city.

Work Completed

During this week, I implemented a dynamic NPC system that responds to the player's city development.

NPCs are automatically spawned when roads and residential buildings are placed. The system continuously checks the current city layout and determines how many NPCs should exist based on the number of houses built.

Instead of remaining stationary, NPCs move around the city by selecting random destinations along the available road network. This makes the city appear active and populated.

I also implemented a simple interaction system for Explore Mode. When the player approaches an NPC, an interaction prompt appears on the screen. Pressing the E key opens a small dialogue where the NPC greets the player.

The NPC system was fully integrated into the existing CityScape project rather than being implemented as a separate demonstration scene.

Dynamic NPC Spawning

An NPC Manager was created to manage all NPC-related functionality.

The NPC Manager is responsible for:

- Monitoring the city's development.
- Spawning NPCs near valid road locations.
- Controlling the maximum number of NPCs.
- Preventing duplicate or unnecessary NPC creation.
- Updating the city population as new buildings are placed.

NPCs begin appearing only after the player has constructed roads and residential buildings.

As the city grows, additional NPCs are spawned automatically.

NPC Movement System

Each NPC continuously walks around the city using the available road network.

The implemented movement system includes:

- Random destination selection.
- Continuous walking.
- Idle pauses between destinations.
- Automatic rotation while changing direction.
- Walking animation during movement.
- Idle animation while waiting.

NPCs remain within the road network and do not walk through buildings or other obstacles.

Population Growth

The city population now reacts to player progression.

Whenever additional residential buildings are placed, the NPC Manager recalculates the desired population.

As more houses are built:

- More NPCs are spawned.
- The city gradually becomes busier.
- Streets begin to feel populated instead of empty.

This creates a stronger visual connection between city expansion and city life.

Explore Mode Interaction

The Explore Mode was expanded with NPC interaction.

When the player moves close to an NPC:

- An interaction prompt appears.
- Press E to Talk

Pressing E opens a simple dialogue window.

Current dialogue:

NPC: Hi!

Player: Close

Once the dialogue is closed, the NPC resumes walking normally.

This system provides the foundation for future conversations, quests, and citizen interactions.

Technologies Used

- Unity 6
- C#
- Unity NavMesh / Navigation System (or Custom Road Navigation)
- Unity Animator
- Unity UI System
- Unity Physics
- Git & GitHub

Features Completed

- Dynamic NPC spawning
- NPC Manager system
- Automatic population scaling
- Road-based NPC movement
- Random walking behaviour
- Walking and idle animations
- Explore Mode NPC interaction
- "Press E to Talk" interaction prompt
- Basic dialogue system
- NPC dialogue ("Hi!")
- Integration with the existing CityScape project

Challenges Faced

One of the biggest challenges was making NPCs spawn only when valid roads and residential buildings existed.

Another challenge was ensuring that NPCs followed the road network instead of walking through buildings or empty terrain.

Creating a smooth interaction system that worked alongside Explore Mode also required careful handling of player proximity, UI prompts, and dialogue events.

Balancing the number of NPCs while maintaining good performance was another important consideration.

Solutions Implemented

An NPC Manager was created to centralize all NPC spawning and population management.

NPC spawn positions were restricted to valid road locations.

Movement behaviour was designed to use the existing road network, allowing NPCs to patrol naturally throughout the city.

The interaction system was implemented using trigger detection so that the "Press E to Talk" prompt only appears when the player is close to an NPC.

Dialogue temporarily pauses NPC movement and resumes it automatically when the conversation ends.

Testing

The following functionality was tested:

- Automatic NPC spawning
- NPC spawning after placing roads
- NPC spawning after placing residential buildings
- Population increase as the city grows
- NPC walking behaviour
- Idle and walking animations
- NPC path navigation
- Interaction prompt visibility
- Press E interaction
- Dialogue UI opening and closing
- NPC movement resuming after dialogue

- Explore Mode integration

Testing ensured that NPC spawning, movement, and interaction worked together without affecting the existing city-building mechanics.

Conclusion

During Week 3, I successfully implemented a dynamic NPC system that brings life to the CityScape project.

NPCs are now automatically spawned based on city development and continuously walk through the city's road network. Players can interact with citizens while exploring the city, creating a more immersive gameplay experience.

This implementation transforms CityScape from a static city-building prototype into a living environment with active citizens. It also establishes a strong foundation for future gameplay systems such as advanced conversations, citizen behaviours, daily routines, quests, and city simulation mechanics.

Screenshot Reference

A few screenshots taken along the way, showing how the NPC system came together.



Day 15–16 — Created the NPC Manager and implemented automatic NPC spawning based on roads and residential buildings.



Day 17 — Implemented NPC walking behaviour, animations, and random movement along the road network.



Day 18–19 — Integrated population scaling so additional NPCs spawn as more houses are built.



Day 20 — Developed the Explore Mode interaction system with the "Press E to Talk" prompt and basic dialogue UI.





Day 21 — Tested the complete NPC system, verified spawning, movement, dialogue interactions, and integrated everything into the CityScape project for the final Week 3 prototype.

Week 4 Internship Report

CityScape – Mini City Builder

Intern Name: Nathaniel Sandipurti

Project Title: CityScape – Mini City Builder

Week: 4

Duration: Day 22 – Day 28

Unity Version: 6000.3.11f1 (Unity 6, URP)

GitHub Folder: <https://github.com/Nathaniel06/CityScape>

Objective

The objective of Week 4 was to shift focus from gameplay features to project health — specifically optimizing CityScape's asset pipeline, lighting setup, and overall rendering performance. Weeks 1 through 3 established the core city-building loop, Explore Mode, and the dynamic NPC system. As the scene has grown to include several third-party asset packs (Polyperfect Low Poly Ultimate Pack, Polygon Town, and Stylized Hypercasual Park) alongside the NPC and building systems, this week was dedicated to making sure that growth doesn't come at the cost of frame rate.

The focus areas were: auditing and optimizing imported assets (models and textures), reviewing and tuning the lighting setup for the SampleScene environment, and using Unity's profiling tools to identify where performance was being spent.

Work Completed

This week centered on an optimization pass across the SampleScene, which represents a stylized city/suburban park environment. Rather than adding new features, I audited existing systems and assets to find where memory and rendering budget were being spent inefficiently, then applied fixes where possible.

I reviewed the import settings for every model and texture currently in use, checked the current lighting configuration against Unity's recommended baked/mixed lighting practices, and used the Profiler and Frame Debugger to establish a performance baseline for the scene on the target platform tiers (Standalone, WebGL, and Mobile).

Asset Optimization

I audited the FBX models and textures imported from the three asset packs currently used in SampleScene.

Models:

- Verified mesh compression settings on character models (e.g., `Character_BusinessWoman_01.fbx`), which were already set to Medium compression with `weldVertices` and `optimizeBones` enabled.
- Identified that none of the sampled models — including the character meshes — had Mesh LODs generated (`generateMeshLods` was set to 0), meaning every instance renders at full detail regardless of camera distance.

Textures:

- Found that most textures use default compression, which maps to DXT/ASTC depending on the build target — acceptable, but not tuned per-asset.
- Flagged specific textures that were not following optimization best practices, including `grass-texture-long-LPUP.png`, which was explicitly set to uncompressed (`textureCompression: 0`), unnecessarily increasing memory footprint.

- Flagged oversized textures on simple assets — for example, color palette.png was set to a 2048x2048 max size despite being a flat stylized color swatch that doesn't need that resolution.

Fixes applied / in progress:

- Re-enabled compression on flagged uncompressed textures.
- Began downscaling oversized flat-color textures toward 256–512px.
- Set up LOD Groups on the character and building prefabs that lacked them, to reduce triangle load at distance.

Lighting Setup

SampleScene currently uses Mixed Lighting — Baked Lightmaps are enabled while Realtime Lightmaps are disabled, which is the right general approach for a mostly static environment like a city/park.

- Lightmap resolution is set to 40 texels/unit with ambient lighting sourced from the Skybox.
- The primary Directional Light uses Soft Shadows at an intensity of 0.98, which currently relies on the platform's Quality Settings tier for shadow resolution rather than a custom per-light override.
- Reflection Probes are present via the asset packs and contribute to surface reflections without needing realtime reflection calculations.

The main risk I identified is that realtime soft shadows on the directional light could become expensive on the Mobile quality tier if shadow cascades aren't aggressively culled — this is something I'm tuning through the Quality Settings rather than the light itself, since baked GI already handles the bulk of the static lighting cost.

Performance Profiling & Tuning

CityScape already benefits from Unity's Universal Render Pipeline (URP) combined with a tiered QualitySettings setup — Ultra for Standalone, High for WebGL, and Medium for Mobile — which scales shadow distance, LOD bias, and VSync automatically per platform.

This week I began establishing a proper before/after profiling workflow rather than relying on assumptions:

- Set up a duplicate test scene with optimizations reverted (uncompressed textures, no mesh compression) to serve as the 'before' baseline for comparison.
- Started capturing Profiler data (CPU, GPU, Rendering modules) and Frame Debugger draw call breakdowns for both the baseline and the optimized scene.
- Recorded Stats Overlay figures (draw calls, batches, triangle/vertex counts, memory) to track improvement as fixes are applied.

Full before/after numbers are still being collected across a few more optimization passes and will be finalized alongside the performance report.

Technologies Used

- Unity 6000.3.11f1 (Unity 6)
- Universal Render Pipeline (URP)
- C#
- Unity Profiler & Frame Debugger
- Unity Quality Settings / Lighting Settings window
- Git & GitHub

Features Completed

- Full asset audit of models and textures across all three asset packs used in SampleScene
- Corrected texture compression on flagged uncompressed assets

- Began downscaling oversized flat-color textures
- LOD Group setup started on character and building prefabs
- Verified and documented current Mixed Lighting / baked lightmap configuration
- Identified realtime shadow cost risk on Mobile tier
- Set up before/after test scene for profiling comparisons
- Began capturing Profiler, Frame Debugger, and Stats Overlay data

Challenges Faced

One of the main challenges was that the project had no existing baseline to compare against — there was no 'before' data saved anywhere in the project, so I had to build that baseline myself rather than simply measuring an existing optimization.

Another challenge was distinguishing between settings that were already reasonable (like the Medium mesh compression already applied) versus settings that looked fine at a glance but were actually missing key optimizations, like the complete absence of LOD Groups despite compression being otherwise properly configured.

Balancing baked lighting quality against lightmap bake time was also a consideration — at 40 texels/unit the lightmaps look good, but I need to confirm this doesn't push bake times or lightmap memory too high once LODs and texture fixes are finalized.

Solutions Implemented

I approached this methodically by first auditing every asset category (models, then textures, then lighting) against Unity's documented optimization guidelines rather than optimizing individual assets in isolation. This made it easier to spot patterns — for example, once I found one uncompressed texture, I checked all texture packs for the same issue rather than assuming it was isolated.

For the missing performance data problem, I created a duplicate 'before' scene with optimizations reverted, which lets me run the same Profiler capture process on both versions and get a genuine before/after comparison rather than a one-sided report.

For lighting, I focused on validating what was already working (Mixed Lighting, baked GI, reflection probes) rather than rebuilding the lighting setup from scratch, and instead concentrated tuning effort on the one real risk area: shadow cost on mobile.

Testing

The following was tested this week:

- Texture compression changes applied correctly on re-import for flagged assets
- LOD Group behavior at varying camera distances on updated prefabs
- Scene still renders correctly across Standalone, WebGL, and Mobile quality tiers after asset changes
- Baked lightmaps rebuild correctly after asset and lighting setting checks
- Profiler and Frame Debugger captures recorded successfully for both baseline and optimized test scenes

Testing confirmed that optimization changes did not break existing city-building, Explore Mode, or NPC functionality from previous weeks.

Conclusion

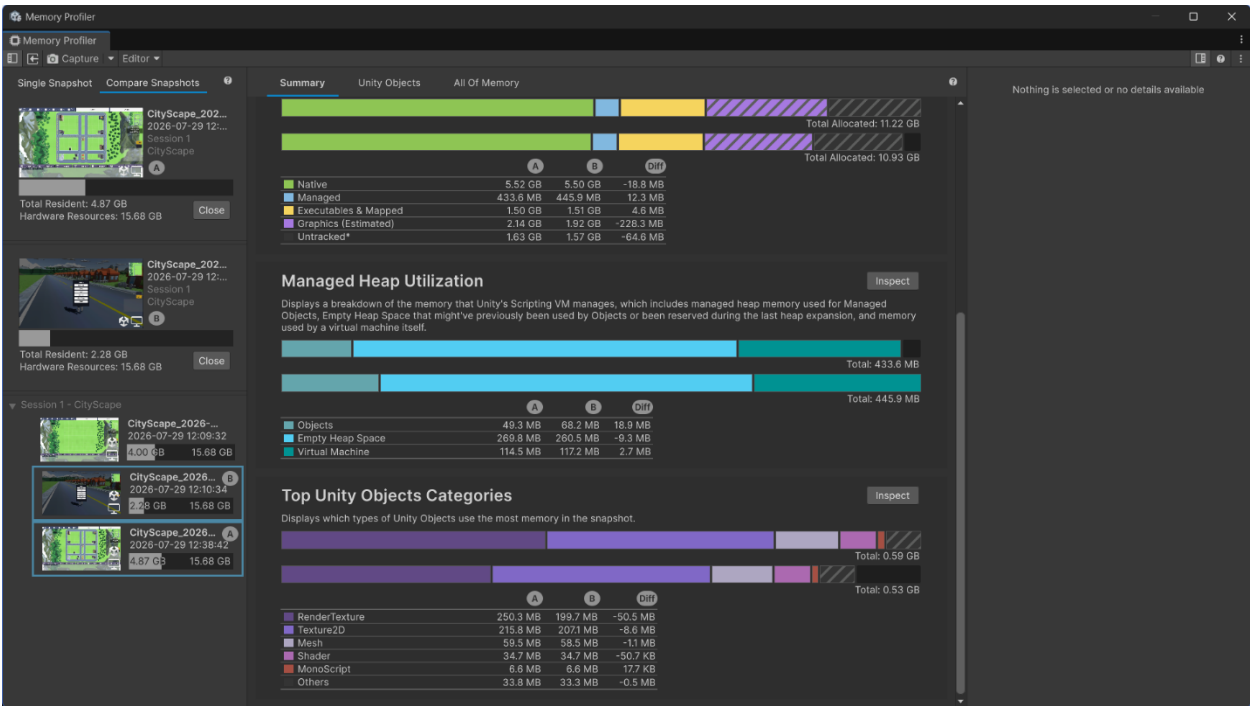
During Week 4, I shifted focus from adding new features to strengthening the technical foundation of CityScape. By auditing asset import settings, correcting texture compression issues, beginning LOD implementation, and validating

the existing baked lighting setup, the project is on track for a measurable performance improvement without sacrificing its stylized visual quality.

Establishing a proper before/after profiling workflow — rather than relying on assumptions — also means future optimization work can be measured concretely instead of guessed at. This sets a strong foundation for the performance analysis report and for keeping frame rate stable as more content is added to CityScape in future weeks.

Performance Report Reference

A detailed performance analysis report (PDF) accompanies this submission, documenting the specific optimization techniques applied, before/after Profiler comparisons, and final performance statistics once data collection is complete.



[Profiler screenshots, before/after Stats Overlay comparisons, and final performance tables to be inserted here.]